

SHHH!

Búsqueda por Voz: Herramienta para VideoJuegos

Propuesta de Mejoras

Autores: Sheila Julvez López · Jose Moreno Barbero · Pablo Cerrada Vega

Asignatura: Usabilidad y Análisis de Videojuegos — Curso 2025-2026

Grado en Desarrollo de Videojuegos · Facultad de Informática · UCM

Introducción

Este documento recoge las mejoras sustanciales propuestas para la reentrega del proyecto SHHH!, una herramienta de control por voz para videojuegos desarrollada con Unity. Las mejoras se han diseñado de manera que puedan ser distribuidas equitativamente entre los tres miembros del equipo (Sheila, Jose y Pablo), maximizando el impacto en las áreas de usabilidad, accesibilidad y robustez técnica.

Cada mejora incluye una descripción detallada, la justificación de su importancia para la asignatura, los pasos de implementación y la asignación de responsabilidades.

Mejora 1 — Estudio de Usabilidad con Usuarios Reales

Responsable principal: **Sheila Julvez**

Justificación

Es la mejora más importante dado el contexto de la asignatura. Teniendo en cuenta la corrección del año pasado, en la que se mencionó que era necesario hacer un estudio con personas reales para verificar el correcto funcionamiento de la herramienta, la mejora propuesta sería añadir un estudio formal con usuarios que convierta el proyecto de una herramienta técnica en un trabajo completo de diseño centrado en el usuario.

Descripción de la mejora

Se diseñará y ejecutará un estudio de usabilidad con un mínimo de 5 participantes, idealmente incluyendo al menos dos personas con discapacidad motriz (público objetivo del sistema). El estudio medirá de forma objetiva la efectividad, eficiencia y satisfacción del sistema de control por voz.

Métricas a recoger

- Tasa de éxito de reconocimiento: porcentaje de comandos pronunciados que se ejecutan correctamente en el primer intento.

- Tiempo de aprendizaje: cuánto tarda un usuario nuevo en dominar el vocabulario de comandos.
- Tasa de errores: frecuencia de comandos mal interpretados o no reconocidos.
- SUS (System Usability Scale): cuestionario estándar de 10 preguntas que genera una puntuación de usabilidad entre 0 y 100.
- NASA-TLX: mide la carga cognitiva percibida por el usuario durante el uso del sistema.

Protocolo de la sesión

- Bienvenida y firma del consentimiento informado (5 min).
- Explicación del sistema sin mostrar los comandos disponibles, para medir la curva de aprendizaje (5 min).
- Tarea guiada: el usuario debe completar las cuatro escenas usando solo voz (15-20 min).
- Cuestionarios SUS y NASA-TLX al finalizar (5 min).
- Entrevista breve semiestructurada: ¿Qué te ha frustrado? ¿Qué mejorarías? (5 min).

Análisis de resultados

Los datos cuantitativos (tasa de éxito, SUS, NASA-TLX) se analizarán con estadística descriptiva. Los datos cualitativos de las entrevistas se agruparán por temas recurrentes. Los resultados alimentarán directamente las otras mejoras: si los usuarios fallan siempre en el mismo comando, se añadirán sinónimos para ese comando; si todos se quejan del feedback visual, se priorizará su mejora.

Mejora 2 — Flexibilidad Semántica de Comandos

Responsable principal: Jose Moreno

Justificación

Actualmente el sistema exige que el usuario pronuncie la frase exacta configurada en el JSON de comandos. Si dice "avanza" en lugar de "move", o "girar" en lugar de "look", el sistema no responde. Esto genera frustración y reduce drásticamente la usabilidad, especialmente para usuarios no nativos en inglés o para personas que tienden a usar sinónimos naturales.

Descripción de la mejora

Se ampliará el sistema de definición de comandos para que cada comando pueda tener asociado un array de frases equivalentes además de la frase principal. El VoiceCommandManager comprobará si la frase detectada coincide con cualquiera de las variantes del comando.

Cambios técnicos necesarios

- Modificar VoiceCommandDefinition para incluir un campo 'alias' (array de strings) junto al campo 'command' existente.

- Actualizar el archivo `commands.json` para añadir los alias de cada comando. Por ejemplo: `move` → `["walk", "forward", "adelante", "avanzar"]`.
- Modificar `VoiceCommandManager` para que al buscar un comando recibido, compruebe tanto el nombre principal como todos sus alias.
- Actualizar `VoiceLoader` para que al inicializar el `VoiceInputEngine`, registre todas las frases (principal + alias) en el reconocedor.
- Actualizar `VoiceCommandEditorWindow` para permitir añadir y eliminar alias desde la interfaz visual del editor.

Alias propuestos para el juego de prueba

- `move` → `walk`, `forward`, `go`, `adelante`, `avanzar`
- `look` → `turn`, `rotate`, `mirar`
- `pick` → `grab`, `take`, `coger`, `recoger`
- `go` → `travel`, `navigate`, `ir`
- `help` → `commands`, `ayuda`, `comandos`
- `quit` → `exit`, `leave`, `salir`

Resultado esperado

La tasa de éxito de reconocimiento debería aumentar significativamente, ya que el sistema acepta un vocabulario mucho más amplio y natural. Esto reduce la curva de aprendizaje y mejora especialmente la experiencia de usuarios hispanohablantes.

Mejora 3 — Comandos Contextuales y Evaluación comparativa

Responsable principal: Pablo Cerrada

3A — Comandos Contextuales

Justificación

Actualmente todos los comandos están activos en todo momento independientemente de la escena o el estado del juego. Esto aumenta la ambigüedad del reconocedor y puede provocar que un comando pronunciado en el menú principal dispare una acción de movimiento, o viceversa. Limitar los comandos disponibles según el contexto reduce errores y hace el sistema más predecible.

Implementación

- Añadir un sistema de contextos al `VoiceCommandManager`: un enum o string que representa el estado actual del juego (`MENU`, `IN_GAME`, `HELP_SCREEN`, `MICROPHONE_CONFIG`, etc.).
- En `VoiceCommandDefinition`, añadir un campo `'contexts'` (array) que indica en qué contextos es válido ese comando.

- VoiceCommandManager filtrará los comandos disponibles según el contexto activo antes de buscar coincidencias.
- GameManager notificará al VoiceCommandManager cada vez que cambie el estado del juego para actualizar el contexto activo.

Ejemplo de distribución de comandos por contexto

- MENU: play, quit, microphone, help, voice, back
- IN_GAME: move, look, pick, coin, help, quit, exit, go
- MICROPHONE_CONFIG: back, exit

3B — Evaluación comparativa de las mejoras 2 y 3A

Objetivo de la evaluación

Además del estudio de usabilidad general, se realizará una prueba específica para evaluar de forma objetiva si la **Flexibilidad Semántica** y los **Comandos Contextuales** mejoran el rendimiento del sistema.

La métrica principal será la **tasa de éxito**, entendida como el porcentaje de acciones solicitadas que el usuario consigue completar correctamente mediante comandos de voz.

Versiones a comparar

Se prepararán tres versiones del sistema:

1. **Versión base:** sin flexibilidad semántica y sin comandos contextuales.
2. **Versión con Flexibilidad Semántica:** incluye alias, pero no comandos contextuales.
3. **Versión con Comandos Contextuales:** incluye filtrado por contexto, pero no alias.

Participantes

Esta prueba se realizará con varias personas. No se utilizarán los mismos participantes que formen parte del estudio principal de la Mejora 1, para evitar sesgos derivados del aprendizaje previo.

Se intentará repetir la prueba con un número razonable de participantes, preferiblemente entre **4 y 6 personas**, dependiendo de la disponibilidad.

Tarea común para todas las versiones

Todos los participantes realizarán exactamente la misma prueba en cada versión del sistema.

La tarea propuesta será:

1. Iniciar el juego mediante voz desde el menú principal.
2. Acceder a una escena jugable.
3. Moverse por el escenario.
4. Girar o mirar en una dirección concreta.
5. Recoger o intentar recoger un objeto.
6. Consultar la ayuda.

7. Cambiar de escena o volver al menú.
8. Salir del juego o cerrar la prueba.

La tarea será la misma en todas las versiones para poder comparar los resultados de forma justa.

Comparaciones previstas

Se compararán los resultados entre:

- Versión base vs. Flexibilidad Semántica.
- Versión base vs. Comandos Contextuales.
- Flexibilidad Semántica vs. Comandos Contextuales.

Resultado esperado

Se espera que:

- La **Flexibilidad Semántica** y los **Comandos Contextuales** mejoren la tasa de éxito.
- La comparación permita justificar con datos qué mejora aporta más valor desde el punto de vista de la usabilidad.

Resumen de tareas por miembro

Tarea	Responsable
SHEILA JULVEZ — Estudio de Usabilidad	
Diseño del protocolo de la sesión de usuarios	Sheila
Reclutamiento de participantes (mínimo 5)	Sheila
Ejecución de las sesiones de prueba	Sheila
Análisis cuantitativo: SUS, NASA-TLX, tasa de éxito	Sheila
Análisis cualitativo: entrevistas semiestructuradas	Sheila
Redacción del informe de resultados del estudio	Sheila
JOSE MORENO — Flexibilidad Semántica	
Modificar VoiceCommandDefinition para soportar alias	Jose
Actualizar commands.json con alias por comando	Jose
Modificar VoiceCommandManager para buscar en alias	Jose
Actualizar VoiceLoader para registrar todas las frases	Jose
Actualizar VoiceCommandEditorWindow con gestión de alias	Jose

Pruebas de reconocimiento con las nuevas variantes	Jose
Preparar la versión del sistema con solo Flexibilidad semántica	Jose
PABLO CERRADA — Comandos Contextuales y Evaluación Comparativa	
Implementar sistema de contextos en VoiceCommandManager	Pablo
Añadir campo 'contexts' en VoiceCommandDefinition y JSON	Pablo
Integrar notificación de contexto desde GameManager	Pablo
Preparar la versión del sistema con solo Comandos Contextuales.	Pablo
Diseñar la prueba comparativa entre versiones	Pablo
Registrar y organizar los datos de tasa de éxito de las versiones comparadas	Pablo
Colaborar en el análisis final de resultados	Pablo
